MATHEMATICAL THEORY / EXACT COMPUTATION

Exact Functional Decomposition of Polynomials with Algebraic Coefficients

A constructive theory and a Wolfram Language reference package

---

Central result. For every proper divisor d of the degree of a polynomial over a characteristic-zero field, there is at most one monic, zero-constant right component of degree d. Its coefficients are recovered by field arithmetic from the leading coefficients of the input. Exact residuals then decide whether it is a genuine component. No polynomial factorization, numerical root finding, or extension of the coefficient field is required.

Deliverables. A proved decomposition algorithm; a complete-decomposition enumerator; exact success and rejection certificates; a Wolfram Language package; 53 Wolfram test specifications; and an independently executed Python/SymPy validation program.

Validation boundary. The independent exact-field checks were executed successfully. The connected Wolfram evaluator returned HTTP 404, so the Wolfram test suite has *not* been executed in a Wolfram kernel in this environment. The software should be run through that suite before production use.

Version 1.0.0 7 September 2026

Abstract

We develop an exact, factorization-free solution to functional decomposition of univariate polynomials whose coefficients are algebraic numbers, whether written as radicals, polynomial `Root` objects, or `AlgebraicNumber` objects. The elementary construction combines a normalized root at infinity with triangular recovery of an outer polynomial. Complete proofs establish uniqueness for a prescribed right degree, soundness and completeness of the decision procedure, descent to the coefficient field, termination of complete decomposition, and exhaustive enumeration modulo affine insertions. The theory is independent of algebraic-number notation and applies to any characteristic-zero field with effective arithmetic; the supplied package intentionally restricts input to recognized exact algebraic numbers.

The accompanying implementation uses coefficientwise `RootReduce`, dense coefficient arrays, a quadratic arithmetic-cost candidate test, and an independent recomposition routine. It distinguishes failure to validate input, a proved absence of a decomposition, and incomplete enumeration caused by an explicit output limit. Worked examples include the motivating radical sextic, its degree-24 composition, quintic algebraic coefficients, repeated derivative factors, Chebyshev collisions, and exact perturbations that defeat numerical zero tests. We also explain why the derivative-factor method is mathematically valid when implemented exhaustively, identify concrete defects in the supplied implementation, and separate arithmetic-operation bounds from the potentially substantial cost of exact number-field arithmetic. The underlying characteristic-zero ideas are classical; the contribution here is a self-contained derivation, a carefully specified implementation, and reproducible validation materials.

Contents

1 The problem and its exact interpretation

1.1 Functional composition, not multiplicative factorization

Let K be a field of characteristic zero. For $f, g \in K[x]$, write

$$f \circ g = f(g(x)).$$

A polynomial p of degree at least two is *decomposable* if $p = f \circ g$ with $\deg f, \deg g \geq 2$. Otherwise it is *indecomposable*. A *complete decomposition* is a list $(f_1, \ldots, f_r)$ of indecomposable polynomials such that

$$p = f_1 \circ f_2 \circ \cdots \circ f_r.$$

The first polynomial is outermost. Composition is associative but generally not commutative.

Multiplicative and functional factorization answer different questions. For example,

$$x^4 + 1 = (x^2 + 1) \circ x^2$$

is a functional decomposition even though $x^4 + 1$ is irreducible over $\mathbb{Q}$ as a product. Conversely $x^4 + x = x(x^3 + 1)$ factors multiplicatively but will be shown to be functionally indecomposable over every characteristic-zero extension of $\mathbb{Q}$.

The motivating question is Reshetnikov's 2019 question about `Decompose` with radical coefficients, together with Hurst's derivative-factor implementation and Lichtblau's historical explanation [1]. Wolfram's documentation describes `Decompose` as a functional decomposition operation and notes nonuniqueness; its documented options currently include `Modulus`, not an `Extension` interface [7]. This article does not assume that the behavior of any particular current kernel is identical to its 2019 behavior. Instead it supplies an independent algorithm.

1.2 The motivating polynomial

Put $s = \sqrt{2}$ and

$$\begin{aligned} p(x) = {} & 3 + 3s + (14 + 4s)x + (12 + 26s)x^2 \\ & + (56 + 8s)x^3 + (8 + 48s)x^4 + 48x^5 + 16sx^6. \end{aligned} \tag{1}$$

The decomposition requested in the question is correct. In the normalization used by our package it becomes

$$h(x) = x^2 + \frac{x}{s}, \tag{2}$$

$$g(x) = 3 + 3s + (8 + 14s)x + (8 + 24s)x^2 + 16sx^3, \tag{3}$$

$$p = g \circ h.$$

There is no disagreement with a right component $x + sx^2$: that component equals $sh(x)$, and the compensating scaling is absorbed into the outer polynomial.

1.3 Coefficient domain and representation

The implemented domain is the embedded algebraic closure $\overline{\mathbb{Q}} \subset \mathbb{C}$, restricted to exact values that the Wolfram kernel recognizes as algebraic. Given

$$p(x) = a_n x^n + \cdots + a_0, \qquad a_n \neq 0,$$

its *coefficient field* is $K = \mathbb{Q}(a_0, \ldots, a_n)$. This is a number field, although an implementation need not explicitly construct a primitive element for it.

A radical and a polynomial `Root` object are two possible descriptions of an exact algebraic value. Algebraic relations among coefficients matter. For example, the values $\sqrt{2}$ and $-\sqrt{2}$ must not be replaced by two unrelated indeterminates satisfying separate quadratic equations. Root isolation data, or an equivalent exact representation, selects the intended embeddings. The general Wolfram `Root` constructor can also represent nonalgebraic roots; the package does not accept a coefficient merely because its head is `Root` [8].

No floating-point coefficient is accepted, even when it is very close to an algebraic number. Inferring an exact value from a decimal approximation is a different, inherently assumption-dependent problem. Constants and linear polynomials are returned as singleton lists by convention; they are not called indecomposable nonlinear factors.

2 Normalization and uniqueness at a fixed right degree

2.1 Removing the affine ambiguity

For an affine polynomial $\ell(x) = ux + v$, $u \neq 0$,

$$f \circ g = (f \circ \ell^{-1}) \circ (\ell \circ g).$$

Without a normalization, even a single decomposition generates infinitely many syntactically different answers.

Definition 2.1. A right component h is *normalized* if it is monic and $h(0) = 0$. A complete chain is normalized if all its components except the first have these properties.

Lemma 2.2 (Affine normalization)**.** *Suppose $p = F \circ H$, where $\deg F = m \geq 2$ and $\deg H = d \geq 2$, over a field extension L/K. Let $c = \mathrm{lc}(H)$ and $b = H(0)$. Then*

$$h = \frac{H - b}{c}, \qquad g(t) = F(ct + b)$$

satisfy $p = g \circ h$, with h normalized. Moreover $\mathrm{lc}(g) = \mathrm{lc}(p)$.

Proof. Substitution gives $g(h(x)) = F(c(H(x) - b)/c + b) = F(H(x))$. The assertions about h follow immediately from its definition. Since h is monic, the leading coefficient of $g(h)$ equals that of g. □

The degrees obey $n = md$. Consequently every proper right degree belongs to

$$\mathcal{D}(n) = \{d \in \mathbb{Z} : d \mid n,\ 1 < d < n\}.$$

In particular $d \leq n/2$. It is essential to test *all* such divisors, not merely prime divisors: an indecomposable polynomial can have composite degree.

2.2 The triangular leading-coefficient argument

Write

$$h(x) = x^d + c_1 x^{d-1} + \cdots + c_{d-1} x,$$

and

$$g(t) = a_n t^m + g_{m-1} t^{m-1} + \cdots + g_0.$$

All terms of $g(h) - a_n h^m$ have degree at most $n - d$. Thus the coefficients of degrees $n-1, \ldots, n-d+1$ in p come entirely from $a_n h^m$.

Theorem 2.3 (A unique normalized candidate)**.** *Fix $n = md$, $m, d \geq 2$, and $p \in K[x]$ of degree n. There is a unique normalized polynomial $h_d \in K[x]$ of degree d satisfying*

$$\deg(p - a_n h_d^m) \leq n - d. \tag{4}$$

Every normalized right component of p of degree d, over any *field extension of K, equals this h_d.*

Proof. Suppose $c_1, \ldots, c_{j-1}$ have already been determined, and put

$$H_{j-1} = x^d + c_1 x^{d-1} + \cdots + c_{j-1} x^{d-j+1}.$$

In the expansion of

$$(H_{j-1} + c_j x^{d-j} + \text{lower terms})^m,$$

the coefficient of x^{n-j} equals

$$[x^{n-j}]H_{j-1}^m + mc_j.$$

Indeed a term involving c_j once and leading terms in all other factors has exactly that degree. Using c_j twice, using any lower coefficient, or combining c_j with a nonleading term produces strictly smaller degree. Therefore

$$c_j = \frac{1}{m}\left(\frac{a_{n-j}}{a_n} - [x^{n-j}]H_{j-1}^m\right), \qquad 1 \leq j < d. \tag{5}$$

The recursion both constructs the desired polynomial and proves its uniqueness. All arithmetic is in K. In any extension field, the same coefficient comparisons impose exactly the same coefficients. Finally a genuine normalized right component satisfies (4) by the preceding degree estimate. □

Notice what the theorem does *not* say. Every proposed degree produces a candidate; most candidates are not components. The lower coefficients of p are needed to accept or reject it. The theorem says only that there is nothing else of that degree to search for.

Corollary 2.4. *There are at most $\tau(n) - 2$ normalized proper right components of a degree-n polynomial, where $\tau(n)$ is the number of positive divisors of n.*

Proof. There is at most one for each member of $\mathcal{D}(n)$. □

These ideas belong to the classical characteristic-zero, or more generally tame, decomposition theory associated with Kozen–Landau and von zur Gathen [2, 3]. Our proofs are included so that the package can be understood without a general decomposition theorem or a polynomial factorization subroutine.

3 A quadratic-cost recurrence from a formal root at infinity

3.1 Reversal and the unit formal root

Set $t = 1/x$ and define the reversed, leading-coefficient-normalized polynomial

$$B(t) = \frac{t^n p(1/t)}{a_n} = 1 + b_1 t + \cdots + b_n t^n, \qquad b_i = \frac{a_{n-i}}{a_n}. \tag{6}$$

For a candidate degree d let $m = n/d$ and $\alpha = 1/m$. There is a unique formal power series

$$U(t) = 1 + u_1 t + u_2 t^2 + \cdots \in K[[t]]$$

with $U^m = B$. This follows by comparing coefficients: the new coefficient u_j has coefficient m, which is invertible in characteristic zero.

Proposition 3.1 (Formal-root description)**.** *The candidate of theorem 2.3 is*

$$h_d(x) = x^d \sum_{j=0}^{d-1} u_j x^{-j}, \qquad u_0 = 1. \tag{7}$$

Equivalently, it is the strictly positive-power part of the formal Laurent series $x^d B(1/x)^{1/m}$.

Proof. Equation (4) is equivalent to

$$\left(t^d h_d(1/t)\right)^m \equiv B(t) \pmod{t^d}.$$

The unique unit-series root modulo t^d is $U \bmod t^d$. Multiplying its reversed truncation by x^d yields (7). The upper limit is $d-1$, so the constant coefficient of h_d is zero. □

The constant term of the Laurent series is deliberately discarded. Retaining it would choose a different affine normalization. No analytic branch of a fractional power is selected here: $U(0) = 1$ determines a formal algebraic expansion. In particular we never need to extract an mth root of a_n.

3.2 Deriving the recurrence used in the code

Differentiate $U = B^\alpha$ formally and eliminate the fractional power:

$$BU' = \alpha U B'.$$

Taking the coefficient of t^{j-1} gives

$$j u_j + \sum_{i=1}^{j} (j-i) b_i u_{j-i} = \alpha \sum_{i=1}^{j} i b_i u_{j-i}.$$

Consequently

$$\boxed{u_j = \frac{1}{j} \sum_{i=1}^{j} ((\alpha+1)i - j) b_i u_{j-i}, \qquad 1 \le j < d.} \tag{8}$$

Only $b_1, \ldots, b_{d-1}$ are read. Computing all required coefficients costs $O(d^2)$ operations in K and $O(d)$ temporary storage.

Remark 3.2 (Division by j is a genuine hypothesis)**.** The elementary recursion (5) divides only by m; its uniqueness argument survives in a field of positive characteristic not dividing m. The faster presentation (8) also divides by j. It cannot be transported unchanged to every tame positive-characteristic case. The supplied package is exclusively characteristic zero.

3.3 The first few coefficients

For hand checks, with $\alpha = 1/m$,

$$\begin{aligned}
u_1 &= \alpha b_1, \\
u_2 &= \alpha b_2 + \frac{\alpha(\alpha-1)}{2} b_1^2, \\
u_3 &= \alpha b_3 + \alpha(\alpha-1) b_1 b_2 + \frac{\alpha(\alpha-1)(\alpha-2)}{6} b_1^3, \\
u_4 &= \alpha b_4 + \alpha(\alpha-1)\left(b_1 b_3 + \frac{b_2^2}{2}\right) \\
&\quad + \frac{\alpha(\alpha-1)(\alpha-2)}{2} b_1^2 b_2 + \frac{\alpha(\alpha-1)(\alpha-2)(\alpha-3)}{24} b_1^4.
\end{aligned}$$

These follow either from (8) or by expanding the formal binomial series for $(1+(B-1))^{\alpha}$. Each displayed expression is a rational polynomial in the input coefficient ratios. This makes the absence of any new algebraic extension particularly explicit.

4 Recovering the outer polynomial and producing certificates

4.1 Membership in the polynomial subalgebra $K[h]$

For a fixed normalized h of degree d, the polynomials

$$1, h, h^2, \ldots, h^m$$

are linearly independent: their leading degrees are $0, d, 2d, \ldots, md$. We can therefore test whether p belongs to their span by descending coefficient elimination.

Start with $R = p$ and $G = 0$. For $k = m, m-1, \ldots, 0$, perform

$$A_k = [x^{kd}]R, \qquad G \leftarrow G + A_k t^k, \qquad R \leftarrow R - A_k h^k. \tag{9}$$

The variable t here merely distinguishes the outer polynomial from the input variable. Software returns both as polynomials in the same specified symbol.

Theorem 4.1 (Exact membership test)**.** *The algorithm (9) produces $G \in K[t]$ and $R \in K[x]$ such that*

$$p = G \circ h + R, \qquad [x^{kd}]R = 0 \quad (0 \le k \le m). \tag{10}$$

There exists an outer polynomial g with $p = g \circ h$ if and only if $R = 0$. In that event $g = G$ and it is unique.

Proof. The equality in (10) is an invariant of every subtraction step. Since h^k is monic of degree kd, the step removes the coefficient at degree kd and changes nothing above that degree. Later steps have smaller degree, so the eliminated coefficient stays zero.

If $R = 0$, the identity gives the desired decomposition. Conversely, suppose $p = g \circ h$. Before the first step, the coefficient at degree md is the leading coefficient of g. Subtracting this multiple of h^m leaves a combination of $1, h, \ldots, h^{m-1}$. Repeating proves that all coefficients of g are extracted and the final residual is zero. The linear independence already observed proves uniqueness. Equivalently, if a nonzero polynomial q satisfied $q(h) = 0$, its leading degree would be $d \deg q$, an impossibility. □

4.2 The full fixed-degree decision theorem

Theorem 4.2 (Soundness, completeness, and rejection)**.** *For a degree-n polynomial $p \in K[x]$ and $d \in \mathcal{D}(n)$, construct h_d by (8), then G_d, R_d by (9). The following statements are equivalent:*

1. *p has a decomposition with right degree d over K.*
2. *p has such a decomposition over some field extension of K.*
3. *$R_d = 0$.*

When these conditions hold, (G_d, h_d) is the unique normalized pair. When $R_d \neq 0$, any explicitly nonzero coefficient of R_d, together with the candidate construction, is a rejection certificate for that degree.

Proof. The implication from the first statement to the second is immediate. A normalized decomposition over an extension must use h_d by theorem 2.3; applying theorem 4.1 in the

extension shows $R_d = 0$. If $R_d = 0$, all arithmetic having been performed in K, the identity $p = G_d(h_d)$ is a decomposition over K. Uniqueness follows from the two preceding theorems. Finally a nonzero residual rejects the unique possible normalized right component, and affine normalization rules out any unnormalized alternative of that degree. □

A negative certificate is more than a failed search. It records the only possible candidate, the only possible outer polynomial, and a nonzero remainder. A certificate of indecomposability consists of these rejections for every proper divisor. For prime input degree the empty divisor list is already a degree-theoretic certificate.

4.3 A small, independently checkable certificate

For $p = x^6 + x$ and $d = 2$, the candidate is $h = x^2$, the recovered outer polynomial is $G = t^3$, and the residual is $R = x$. Thus

$$x^6 + x = (x^2)^3 + x.$$

The coefficient of x is exactly 1, not merely numerically distinguishable from zero. The same construction at $d = 3$ gives $h = x^3$, $G = t^2$, and again $R = x$. These two certificates prove absolute indecomposability.

The package returns the largest-degree nonzero residual coefficient as a convenient witness. It does not require the largest coefficient in absolute value, numerical evaluation, or sign determination.

5 Descent, embeddings, and coefficient-field minimality

Corollary 5.1 (Field descent)**.** *Let L/K be any extension of characteristic-zero fields and let $p \in K[x]$. Every polynomial decomposition over L is affinely equivalent to a normalized decomposition over K. In particular, indecomposability of p over K is equivalent to indecomposability over an algebraic closure of K.*

Proof. For two factors, apply affine normalization and theorem 4.2. For a longer chain, normalize the innermost factor, which is a right component of p, and recover its outer cofactor in $K[x]$. Apply the same argument to that cofactor, proceeding outward. If an indecomposable factor acquired a decomposition over an extension, the two-factor result would descend it back to K, a contradiction. □

This is the decisive distinction from ordinary multiplicative factorization. Extending $\mathbb{Q}$ to $\mathbb{Q}(\sqrt{2})$ splits $x^2 - 2$ as a product, but extending the coefficient field cannot create a new normalized right degree for polynomial composition in characteristic zero. A factoring-based method may need an extension option to expose derivative factors; the underlying composition problem does not need one.

Corollary 5.2 (Smallest coefficient field)**.** *If all coefficients of p belong to $K = \mathbb{Q}(a_0, \ldots, a_n)$, every coefficient of every normalized complete decomposition belongs to K.*

Proof. The recursion uses addition, subtraction, multiplication, and division by nonzero elements already in K, and the preceding corollary applies recursively. □

The statement concerns values, not visual notation. A coefficient may print as a complicated `Root` expression even when it belongs to a small field. Conversely a radical expression may syntactically mention larger extensions whose contributions cancel.

Proposition 5.3 (Equivariance under embeddings)**.** *Let $\sigma : K \to L$ be a field embedding. For every proper candidate degree, applying σ to the candidate, recovered outer polynomial, and residual for p gives exactly those constructed for $\sigma(p)$. In particular the set of successful right degrees is unchanged.*

Proof. Both recurrences are built from field operations and rational constants. Embeddings preserve these operations and are injective, so they preserve both zero and nonzero residuals. □

For real algebraic coefficients this shows, or the explicit recurrences show directly, that the normalized components are real. Complex algebraic coefficients are equally valid; no ordering of the field is used.

Remark 5.4 (Fields are not rings)**.** This package does not solve decomposition with coefficients constrained to a ring of integers or another subring. Normalization may introduce denominators: $(2x^2 + x)^2$ has an integral decomposition, but its monic right component is $x^2 + x/2$. Ring-restricted decomposition requires additional integrality and scaling analysis. The literature treats those questions separately [6]. A rational coefficient in a normalized answer is not a certificate that integral decompositions are impossible.

6 Complete decompositions and exhaustive enumeration

6.1 Why the smallest successful right degree is enough

Lemma 6.1. *Let d be the smallest successful proper right degree of p. Then its normalized right component h_d is indecomposable.*

Proof. If $h_d = A \circ B$ with both degrees at least two, then B is a right component of $p = G_d \circ A \circ B$ and $1 < \deg B < d$. Affine normalization does not change this degree. This contradicts minimality of d. □

The function `AlgebraicDecompose` tests proper divisors in increasing order. If none succeeds, it returns the singleton input. Otherwise it appends h_d to a complete decomposition of G_d. It need not recursively decompose h_d: theorem 6.1 has already proved its indecomposability.

Theorem 6.2 (One complete chain)**.** *For every $p \in K[x]$ of degree at least two, this procedure terminates and returns a normalized complete decomposition over K.*

Proof. Every successful split reduces the degree of the recursive input from n to $n/d \leq n/2$. Therefore recursion terminates. If no split succeeds, theorem 4.2 proves that the input is indecomposable. Otherwise the returned inner factor is indecomposable by theorem 6.1, and the induction hypothesis applies to the outer cofactor. Appending the inner factor preserves the composition identity and the normalization of every nonfirst factor. □

The recursion depth is at most $\lfloor \log_2 n \rfloor$. An arbitrary successful split would instead require recursively decomposing *both* factors. Recursing only on the outer factor is justified here by the deliberate minimum-degree choice, not by a general property of arbitrary decompositions.

6.2 Enumerating all normalized complete chains

For enumeration, inspect every successful normalized pair (g, h). Keep the pairs for which h is indecomposable; enumerate every complete decomposition of g and append h. If the input has no proper split, its only complete decomposition is its singleton list.

Theorem 6.3 (Exhaustiveness and absence of duplicates)**.** *This enumeration returns exactly one representative of every complete decomposition modulo insertion of affine maps. Every output is normalized, and no normalized chain occurs twice.*

Proof. Any normalized complete chain has an indecomposable last factor h. This is one of the successful normalized right components listed by theorem 4.2. Its outer cofactor is unique by theorem 4.1, and the remaining prefix is a complete decomposition of that cofactor. Induction therefore generates the chain.

Conversely every appended last factor is indecomposable, every recursively generated prefix is complete, and each chosen pair is a verified composition identity. Thus every output is a complete decomposition. Distinct successful last degrees have distinct last factors. For a fixed last degree, both that factor and its cofactor are unique, and induction excludes duplicate prefixes. Finally normalizing a chain from the inside outward by theorem 2.2 accounts for every affine-equivalence class. □

Corollary 6.4 (At most one chain per ordered degree sequence)**.** *There is at most one normalized complete decomposition with any fixed ordered sequence of component degrees.*

Proof. The last degree determines the normalized last factor; cancellation by that nonconstant factor determines the outer cofactor. Repeat for the remaining sequence. □

This gives an elementary finiteness proof: a degree-n chain has at most $\lfloor \log_2 n \rfloor$ factors, each at least two, whose product is n. There are only finitely many such ordered multiplicative factorizations. The word "all" here means modulo affine insertions, *not* modulo arbitrary Ritt transformations. Different degree orders, such as $x^2 \circ x^3$ and $x^3 \circ x^2$, are deliberately returned separately.

6.3 Output limits must not masquerade as completeness

The enumerator has an option `"MaxDecompositions"`, with default value 1000. It internally retains at most one more chain than that limit. If this extra chain exists, it returns

```
Failure["EnumerationLimit", <|
  "Complete" → False,
  "Limit" → limit,
  "PartialDecompositions" → partial
|>]
```

with an additional explanatory message field. A list is returned only when the enumeration fits within the limit. Use `"MaxDecompositions" -> Infinity` to request unrestricted enumeration.

Keeping at most $L + 1$ chains in recursive subproblems is sufficient. A subproblem with more than L prefixes, attached to any admissible last factor, already forces more than L full chains. If no subproblem exceeds L, accumulating their contributions detects the first excess at the parent. This argument relies on the absence of duplicates established above.

7 The finite right-component poset

The degree uniqueness theorem has a stronger consequence that is useful for understanding the whole decomposition structure. Include the two trivial normalized components

$$h_1 = x, \qquad h_n = \frac{p - p(0)}{a_n},$$

and let $S(p)$ be their degrees together with all successful proper right degrees.

Theorem 7.1 (Divisibility is the right-component order)**.** *For $d, e \in S(p)$, there exists a polynomial q satisfying $h_e = q \circ h_d$ if and only if $d \mid e$. If it exists, q is unique, monic, has zero constant term, and has degree e/d.*

Proof. The forward implication follows from multiplication of degrees, and uniqueness follows from injectivity of substitution into a nonconstant polynomial. Leading coefficients and constant terms give the normalization of q.

For the converse suppose $d \mid e$ and write $p = g_d(h_d)$, where g_d has leading coefficient a_n and degree n/d. Put $m_e = n/e$. The formal Laurent series

$$V(t) = t^{e/d} \left(\frac{g_d(t)}{a_n t^{n/d}} \right)^{1/m_e}$$

has integer exponents, leading coefficient one, and $V^{m_e} = g_d/a_n$. Its composition with h_d is well defined at infinity and has leading term x^e. Uniqueness of a unit formal root gives

$$V(h_d(x)) = x^e B(1/x)^{1/m_e}.$$

Every negative power of h_d has only strictly negative powers of x in its Laurent expansion at infinity. Every positive power of h_d is a polynomial with zero constant term. Therefore the strictly positive-power part of $V(h_d)$ is obtained by taking the strictly positive-power polynomial part q of V and substituting h_d. By theorem 3.1, that positive-power part is h_e. Thus $h_e = q(h_d)$, as required. The argument includes $d = 1$ and $e = n$. □

In particular one can summarize all normalized right components by a finite divisibility poset on at most $\tau(n)$ vertices. Its cover relations encode indecomposable quotients.

Proposition 7.2 (Complete chains as maximal divisor chains)**.** *Let*

$$1 = d_0 \mid d_1 \mid \cdots \mid d_r = n$$

be a strict chain in $S(p)$ and write $h_{d_i} = q_i \circ h_{d_{i-1}}$. Then

$$p = (a_n q_r + p(0)) \circ q_{r-1} \circ \cdots \circ q_1. \tag{11}$$

This is a complete decomposition exactly when every adjacent relation is a cover relation in the divisibility poset. All normalized complete decompositions arise this way.

Proof. Composing the quotient identities yields $h_n = q_r \circ \cdots \circ q_1$, which proves (11). Suppose some quotient $q_i = A \circ B$ is decomposable. Normalize B; since q_i is monic with zero constant term, the adjusted A can also be taken monic with zero constant term. Then $B(h_{d_{i-1}})$ is a normalized right component of p whose degree lies strictly between d_{i-1} and d_i and divides d_i. Thus the relation is not a cover.

Conversely, an intermediate degree $e \in S(p)$ with $d_{i-1} \mid e \mid d_i$ gives two quotient identities by theorem 7.1; their composition factors q_i nontrivially. The outer affine map in (11) does not change decomposability. Finally the suffix compositions of any normalized complete chain are normalized right components of p and provide its associated maximal divisor chain. □

This yields an alternative architecture: compute $S(p)$ once, recover quotient labels on its cover edges, and enumerate maximal paths. The present package uses the simpler recursive algorithm of theorem 6.3; it does *not* expose a poset or graph API. The poset description is included as a proved structural result and a concrete route to a more efficient future enumerator.

7.1 A field-tower interpretation

For any nonconstant polynomial h of degree d,

$$[K(x) : K(h(x))] = d. \tag{12}$$

Here is an elementary proof. The element x satisfies $h(T) - h(x) = 0$ over $K(h(x))$, so the degree is at most d. If $1, x, \ldots, x^{d-1}$ were linearly dependent over $K(h)$, clearing denominators would give a nonzero relation $\sum_{i=0}^{d-1} A_i(h)x^i = 0$ with $A_i \in K[t]$. The nonzero summands have distinct leading degrees modulo d, so their highest terms cannot cancel. This contradiction gives the opposite inequality.

A complete decomposition thus produces a tower

$$K(p(x)) \subset K(f_2 \circ \cdots \circ f_r(x)) \subset \cdots \subset K(f_r(x)) \subset K(x),$$

whose successive degrees are the component degrees. Our algorithms do not construct splitting fields, Galois groups, or these intermediate fields explicitly.

Ritt's classical theory gives stronger global information: complete polynomial decompositions in characteristic zero have the same length and the same multiset of degrees, and are related by elementary neighboring transformations. Zieve and Müller give a modern account and refinements [5]. We use neither that classification nor a Ritt-move search in the correctness proofs above.

8 The derivative-factor approach: valid principle, fragile implementation

8.1 What a complete derivative search would have to do

If $p = g \circ h$, then

$$p'(x) = g'(h(x))h'(x).$$

In characteristic zero, $h' \neq 0$ and hence h' divides p' in the coefficient field. For a normalized component of degree d, h'/d is a monic divisor of p' of degree $d-1$.

Therefore an exhaustive derivative-based algorithm can factor p', enumerate every monic divisor v of degree $d-1$ for every $d \in \mathcal{D}(n)$, set

$$h(x) = d \int_0^x v(t)\, dt,$$

and apply the exact membership test of theorem 4.1. This version is complete: the true divisor h'/d is among the candidates. The coefficient field suffices, by theorem 5.1; there is no need to split every irreducible derivative factor into linear factors.

The obstacle is efficiency. If $p' = c\prod_i v_i^{e_i}$, a multiset-aware enumeration has $\prod_i (e_i + 1)$ possible divisors before degree filtering. Expanding all repeated factors into a list and taking arbitrary subsets is worse, because identical divisors occur many times. The formal-root method considers only one candidate per right degree and performs no derivative factorization.

8.2 Concrete defects in the supplied code

The mathematical identity in the question is correct, but the implementation needs repairs beyond an algebraic extension option.

A single distinct derivative factor does not imply indecomposability. The early return triggered by fewer than two nonconstant factor-list entries is invalid. For

$$p = (x+1)^6, \qquad p' = 6(x+1)^5,$$

there is only one distinct nonconstant derivative factor. Nevertheless

$$p = (t+1)^3\big|_{t=x^2+2x}$$

is a nontrivial decomposition. Multiplicities cannot be discarded from the logical test.

Adjacent power maps compose by multiplying exponents. The displayed rewrite rule replacing consecutive x^n, x^m by x^{n+m} is wrong:

$$x^n \circ x^m = x^{nm}.$$

For example, the chain (x^2, x^3) represents x^6, not x^5. A postprocessing pass containing the addition rule can turn a correct decomposition into a false identity. The supplied package never merges adjacent power factors, because doing so would also destroy completeness when the factors are indecomposable.

Candidate derivative degrees need the right arithmetic filter. One must impose $d = \deg(v) + 1 \mid n$, with $1 < d < n$. Merely bounding $\deg(v)$ by $n/2$ admits invalid nondivisor degrees and even allows $d > n/2$ at the upper endpoint. The proper derivative-degree bound is $d - 1 \leq n/2 - 1$. Exact divisibility checks should precede coefficient-system construction.

Only the outer component is repeatedly decomposed. That strategy is complete only if each accepted inner component has already been proved indecomposable. Subset cardinality in a derivative factor list is not degree order. Thus the supplied search ordering does not itself establish the hypothesis of theorem 6.1. Either recurse on both factors or deliberately choose the smallest successful right degree.

Numerical plausibility is not an exact certificate. A successful coefficient system and an exact algebraic residual test can establish correctness; a heuristic zero test alone should not be advertised as a proof. Degree detection must also follow algebraic coefficient reduction, since a visibly complicated leading coefficient may be exactly zero.

These are separate issues. Adding `Extension` to a factoring operation may help expose candidates, but it cannot repair an incorrect exponent rewrite, an invalid early return, or incomplete recursion.

9 Explicit criteria and worked examples

9.1 A complete quartic criterion

Let

$$p = a_4x^4 + a_3x^3 + a_2x^2 + a_1x + a_0, \qquad a_4 \neq 0.$$

There is only one proper degree, $d = 2$. Define

$$b = \frac{a_3}{2a_4}, \qquad h = x^2 + bx, \qquad c = a_2 - a_4b^2.$$

The recovered outer polynomial and residual are

$$G(t) = a_4t^2 + ct + a_0, \qquad R = (a_1 - bc)x.$$

Therefore

$$\boxed{p \text{ decomposes } \iff 8a_4^2a_1 - 4a_4a_3a_2 + a_3^3 = 0.} \tag{13}$$

This is valid for arbitrary algebraic coefficients, including nonreal coefficients and nonradical `Root` values. It follows by direct expansion, so no polynomial factorization or algebraic equation solving is hidden in the criterion.

9.2 Both possible sextic degree patterns

For $p = \sum_{j=0}^{6} a_jx^j$, $a_6 \neq 0$, the only possible proper right degrees are 2 and 3.

For $d = 2$, put

$$b = \frac{a_5}{3a_6}, \qquad h = x^2 + bx,$$
$$C = a_4 - 3a_6b^2, \qquad D = a_2 - Cb^2.$$

Then

$$G(t) = a_6t^3 + Ct^2 + Dt + a_0,$$
$$R(x) = (a_3 - a_6b^3 - 2Cb)x^3 + (a_1 - Db)x.$$

Thus both displayed residual coefficients must vanish.

For $d = 3$, define

$$u = \frac{a_5}{2a_6}, \qquad v = \frac{a_4}{2a_6} - \frac{a_5^2}{8a_6^2}, \qquad h = x^3 + ux^2 + vx,$$

and $B = a_3 - 2a_6uv$. Then

$$G(t) = a_6t^2 + Bt + a_0,$$
$$R(x) = (a_2 - a_6v^2 - Bu)x^2 + (a_1 - Bv)x.$$

Again the two coefficients give a necessary and sufficient test. Testing only the quadratic inner candidate would miss valid degree pattern $(2, 3)$; testing only the cubic inner candidate would miss pattern $(3, 2)$.

9.3 Detailed recovery of the motivating sextic

For (1) and $d = 2$, $m = 3$ and

$$b_1 = \frac{48}{16s} = \frac{3}{s}, \qquad u_1 = \frac{b_1}{3} = \frac{1}{s}.$$

So (2) is forced. Expanding only what is needed,

$$h^2 = x^4 + sx^3 + \frac{1}{2}x^2,$$

$$h^3 = x^6 + \frac{3}{s}x^5 + \frac{3}{2}x^4 + \frac{1}{2s}x^3.$$

Descending elimination first takes $16s$ at degree six, then $8 + 24s$ at degree four, then $8 + 14s$ at degree two, and finally $3 + 3s$. The residual is zero, yielding (3). Both factors are indecomposable because their degrees are prime.

For completeness, the other possible degree gives

$$\begin{aligned} h_3(x) &= x^3 + \frac{3s}{4}x^2 + \frac{15+2s}{16}x, \\ G_3(t) &= 16st^2 + (11+2s)t + 3 + 3s, \\ R_3(x) &= \frac{3}{16}\left((8+17s)x^2 + (17+4s)x\right). \end{aligned}$$

The residual is nonzero. Hence this sextic has exactly one normalized proper pair and exactly one normalized complete decomposition.

9.4 The degree-24 composition

Let $P(x) = p(x^4 - x + 1)$. The package's normalized chain is

$$\begin{aligned} F_1(x) &= 141 + 105s + (168 + 142s)x \\ &\quad + (56 + 72s)x^2 + 16sx^3, \\ F_2(x) &= x^2 + \left(2 + \frac{1}{s}\right)x, \\ F_3(x) &= x^4 - x. \end{aligned}$$

Thus $P = F_1 \circ F_2 \circ F_3$ and the degree sequence is $(3, 2, 4)$. To see the shift directly, put $k = x^4 - x$ and $c = 1 + 1/s$. Then

$$h(k+1) = k^2 + (2 + 1/s)k + c = F_2(k) + c,$$

so $F_1(t) = g(t + c)$. The quartic $x^4 - x$ fails (13), since its linear coefficient is -1 and its cubic coefficient is zero. The three factors are therefore indecomposable. This example also explains why a prime-divisor-only search is not a complete algorithm.

9.5 A quintic algebraic coefficient with an independent radical

Let

$$\alpha = \mathrm{Root}(z^5 - z - 1, \text{the real root}),$$

represented in Wolfram Language as `Root[##^5-##-1 &,1]`. Take

$$h = x^3 + \alpha x, \qquad g = (1+\alpha)x^2 + \alpha^2 x + \sqrt{3}.$$

Then

$$\begin{aligned} p = g(h) &= (1+\alpha)x^6 + 2\alpha(1+\alpha)x^4 + \alpha^2 x^3 \\ &\quad + \alpha^2(1+\alpha)x^2 + \alpha^3 x + \sqrt{3}. \end{aligned}$$

The degree-two candidate is x^2 and fails because of the nonzero odd terms. The degree-three recurrence gives $x^3 + \alpha x$, and exact recovery gives g. No conversion of α into radicals is needed. The resulting two factors are complete by their prime degrees.

The minimal-polynomial relation $\alpha^5 - \alpha - 1 = 0$ also provides a degree-reduction regression: adding $(\alpha^5 - \alpha - 1)x^9$ to this input must not turn it into a degree-nine problem. Canonical coefficient reduction is done before the list of degree divisors is constructed.

9.6 Monomials, Chebyshev polynomials, and nonuniqueness

For $p = x^n$, every candidate is $h_d = x^d$, and every divisor succeeds. A complete chain consists precisely of prime-degree power maps. If

$$n = \prod_{i=1}^{k} p_i^{e_i}, \qquad s = e_1 + \cdots + e_k,$$

there are exactly

$$\frac{s!}{e_1! \cdots e_k!} \tag{14}$$

normalized complete chains: the distinct orders of the multiset of prime factors. Uniqueness per ordered degree sequence excludes further answers. For x^{12} these are

$$(x^3, x^2, x^2), \quad (x^2, x^3, x^2), \quad (x^2, x^2, x^3).$$

Chebyshev polynomials supply a nonmonomial family. Their defining identity $T_n(\cos\theta) = \cos(n\theta)$ gives $T_m \circ T_d = T_{md}$ by evaluation on infinitely many real points. For every $d \mid n$, the unique normalized right component of T_n is

$$H_d(x) = \frac{T_d(x) - T_d(0)}{2^{d-1}}.$$

All proper divisors therefore succeed, and theorem 7.2 again gives the count (14). In particular

$$\begin{aligned} T_6 &= (32x^3 - 48x^2 + 18x - 1) \circ x^2 \\ &= (32x^2 - 1) \circ \left(x^3 - \frac{3}{4}x\right). \end{aligned}$$

The enumerator returns both normalized chains, not just a preferred one.

9.7 An infinite indecomposable family and tiny perturbations

For every $n \geq 2$ and $c \neq 0$, the polynomial $x^n + cx$ is absolutely indecomposable in characteristic zero. If n is prime, this follows from degrees. Otherwise each proper candidate has $d \leq n/2$, and all the required nonleading top coefficients vanish. Thus $h_d = x^d$, the recovered outer polynomial is $t^{n/d}$, and the residual is $cx \neq 0$. By theorem 4.2 there is no decomposition.

Taking $c = 10^{-100}$ gives an exact polynomial arbitrarily close to the decomposable polynomial x^n, yet still indecomposable. A small numerical residual is not a logically sufficient reason to replace it by zero.

10 Arithmetic complexity and the geometry of decomposition

10.1 A transparent bound for the reference implementation

We count one addition, subtraction, multiplication, division, or equality test in K as one field operation. This is a model of the polynomial algorithm, not a claim that algebraic-number operations take constant machine time.

For a fixed proper right degree d, recurrence (8) uses $O(d^2)$ field operations. Let $m = n/d$. The implementation stores $1, h, \ldots, h^m$, building successive powers by dense convolution. Forming h^k from h^{k-1} costs $O(kd^2)$ operations, so all powers cost

$$O\left(d^2 \sum_{k=1}^{m} k\right) = O(n^2).$$

The descending subtractions require at most $O(mn) = O(n^2/d)$ additional operations. Consequently a fixed-degree attempt costs $O(n^2)$ field operations, with $O(n^2/d)$ field elements of storage for the cached powers. This implementation favors simple, auditable routines rather than optimal memory usage.

Testing every proper degree costs $O(\tau(n)n^2)$ field operations. For one complete chain, the successive recursive degrees n_j divide n and satisfy $n_{j+1} \le n_j/2$. Hence

$$\sum_j \tau(n_j)n_j^2 \le \tau(n)n^2 \sum_{j\ge 0} 4^{-j} = \frac{4}{3}\tau(n)n^2.$$

The same $O(\tau(n)n^2)$ bound therefore applies to finding one complete decomposition, excluding input conversion and the cost of individual field operations. Prime-degree inputs require no candidate tests.

The all-decompositions function is output-sensitive in the unavoidable sense that it must materialize its output chains. It also performs recursive candidate tests and uses call-local memoization. The preceding single-chain bound must not be advertised as a bound for unrestricted enumeration. The output cap controls the number of retained chains per subproblem; it is not a running-time or memory guarantee for all intermediate coefficient arithmetic.

10.2 Number-field arithmetic is a separate cost

If $K = \mathbb{Q}(\theta)$ has degree D, a coefficient can be represented in the basis $1, \theta, \ldots, \theta^{D-1}$ with rational entries. In this representation addition, multiplication modulo the minimal polynomial, and inversion by an extended Euclidean algorithm are exact, but their bit costs depend on both D and the heights of the rational entries. Large composita, expensive exact conversion, and coefficient growth may dominate the polynomial-level complexity.

The package instead uses coefficientwise `RootReduce`. Wolfram documents its reduction of algebraic combinations to canonical algebraic forms, including representations using `Root` [9]. Repeating this conversion is intentionally conservative: it makes the zero invariant visible at every arithmetic boundary, but may reconstruct algebraic information repeatedly. Nothing in the reference implementation promises performance comparable to a specialized number-field library.

A useful future backend would construct one primitive element for the input coefficient field, convert all coefficients once, and keep coordinate vectors in that field throughout. Wolfram's `ToNumberField` and `AlgebraicNumber` provide relevant interfaces [12, 11]. Field descent proves that such a fixed field is sufficient for the entire computation. This backend is a proposed optimization, not an option silently present in version 1.0.0.

Faster polynomial decomposition algorithms also exist. Kozen–Landau's factorization-free work and the subsequent tame algorithms are the appropriate historical baseline, not derivative-subset enumeration [2, 3]. Giesbrecht gives a detailed account of triangular recovery, right division, and faster arithmetic bounds [4]. We make no claim that the recurrence, field-descent theorem, or asymptotic arithmetic complexity is new.

10.3 Equations defining a fixed-degree decomposition locus

The same construction gives more than an algorithm on individual inputs. Work over an algebraically closed characteristic-zero field and consider the space of degree-n polynomials, whose leading coefficient is nonzero. It has dimension $n + 1$.

Proposition 10.1 (Fixed-degree locus)**.** *For $n = md$ with $m, d \ge 2$, the locus of polynomials with a right component of degree d is cut out, within the nonzero-leading-coefficient chart, by the*

residual equations $R_d = 0$. It is isomorphic to the parameter space of pairs

$$g(t) = a_n t^m + \cdots + g_0, \quad a_n \neq 0, \qquad h = x^d + c_1 x^{d-1} + \cdots + c_{d-1} x.$$

In particular it is irreducible, smooth, and has dimension $m + d$ and codimension

$$(n + 1) - (m + d) = (m - 1)(d - 1).$$

Proof. Composition is a polynomial map from the displayed parameter space into the degree-n coefficient chart. It is injective by normalized uniqueness. The candidate coefficients and the descending-recovery coefficients are rational functions of the input coefficients with denominators that are products of powers of a_n and nonzero integers. These are regular functions on the chart $a_n \neq 0$.

The residual coefficients are consequently regular functions on this chart. Their common zero locus is exactly the image of composition by theorem 4.2. On that image the candidate-and-recovery map is a regular inverse. Thus composition is an isomorphism onto the stated locus. The parameter space is a product of affine spaces and a one-dimensional multiplicative group, with $(m + 1) + (d - 1) = m + d$ parameters, giving the remaining assertions. □

Clearing powers of a_n gives explicit polynomial equations, but $a_n \neq 0$ must still be imposed. Otherwise degree drops introduce spurious boundary components. Different right-degree loci can intersect; their union need not be smooth even though each fixed-degree locus is. The quartic equation (13) is the smallest nontrivial illustration. The proposition also explains why random exact perturbations usually destroy a chosen decomposition: each fixed-degree locus has positive codimension.

11 The Wolfram Language package

11.1 Loading and basic use

Keep the extracted directory structure intact. From a notebook or kernel session, load the package by its full path:

```
Get["/path/to/algebraic-polynomial-decomposition/AlgebraicDecomposition.wl"];
Clear[x];
p = 3 + 3 Sqrt[2] + (14 + 4 Sqrt[2]) x +
    (12 + 26 Sqrt[2]) x^2 + (56 + 8 Sqrt[2]) x^3 +
    (8 + 48 Sqrt[2]) x^4 + 48 x^5 + 16 Sqrt[2] x^6;
chain = AlgebraicDecompose[p, x];
VerifyAlgebraicDecomposition[p, chain, x]
```

The root-level file is a loader for `Kernel/AlgebraicDecomposition.wl`; alternatively load that kernel file directly. There are no external package dependencies. All public symbols live in the context `AlgebraicDecomposition`‘. The variable argument must evaluate to an unassigned symbol. As with ordinary Wolfram functions, input expressions are evaluated before they reach the package; the package does not sandbox evaluation or recover an expression already changed by user definitions.

The mathematical result for `chain` is (3) followed by (2). Equivalent exact algebraic coefficients may display differently depending on canonicalization. The package intentionally does not apply `ToRadicals` to all answers: a radical rendering is not needed for correctness, and many algebraic values have no radical expression over $\mathbb{Q}$.

11.2 Public interfaces

One complete decomposition. `AlgebraicDecompose[p,x]` returns a list of factors, outermost first. It tests right degrees in ascending order and recursively decomposes the outer cofactor. For degree at least two the result is a complete chain. Zero, constants, and linear inputs return their singleton convention.

Every proper normalized pair. `AlgebraicRightDecompositions[p,x]` returns all pairs `{g,h}` with $p = g \circ h$ and $1 < \deg h < \deg p$. Pairs are ordered by increasing right degree. Neither member is promised indecomposable. An empty list for a nonlinear input proves absolute indecomposability; an empty list for a constant or a linear input merely means there are no proper pairs.

A specified-degree attempt. `AlgebraicDecompositionAttempt[p,x,d]` requires a proper divisor d of the reduced input degree. The result is an association with the following entries:

Key	Meaning
`InputDegree`, `RightDegree`, `OuterDegree`	The degrees n, d, m.
`Decomposable`	Whether the exact residual is zero for this degree.
`Outer`, `Inner`, `Residual`	G_d, h_d, R_d, satisfying $p = G_d(h_d) + R_d$.
`NonzeroWitness`	The exponent and coefficient of the leading nonzero residual term, or `Missing["NotApplicable"]`.

The returned `Outer` is a candidate even when `Decomposable` is false. A rejection for one degree is not a rejection for every other degree.

Every complete decomposition.

```
AlgebraicDecompositions[p, x]
AlgebraicDecompositions[p, x, "MaxDecompositions" → Infinity]
```

Each element of the returned list is itself a complete chain. The normalization is modulo affine insertions only. The default finite limit is 1000, and exceeding it returns the explicitly incomplete `Failure` described earlier. For constants and linear inputs the answer is a one-element list containing the singleton convention.

Exact composition. `ComposeAlgebraicPolynomials[chain,x]` composes a nonempty list by coefficient-array Horner evaluation. It also accepts constant and linear members. It does not establish completeness: composing a list and deciding whether its factors are indecomposable are different operations.

Verification. `VerifyAlgebraicDecomposition[p,chain,x]` independently recomposes the chain and returns an association. `IdentityVerified` tests $p = f_1 \circ \cdots \circ f_r$ coefficientwise and `Residual` records the difference. `Normalized` checks monicity and zero constant term for all nonfirst factors. `AllFactorsIndecomposable` requires that every factor have degree at least two and pass the complete candidate-degree test. `ValidCompleteDecomposition` is the conjunction of the identity and that indecomposability condition; it does not require normalization. An unnormalized but genuinely complete identity is still a valid complete decomposition.

The additional field `ValidConstantOrLinearConvention` is true only for a correct singleton constant or linear convention. `Degrees` lists component degrees, with $-\infty$ for the zero polynomial.

This avoids the misleading claim that a constant is an indecomposable nonlinear component. The recomposition code is separate from candidate recovery, but the completeness check uses the same mathematical decision routine; this is not a second independently implemented completeness prover.

11.3 Algebraic validation and canonical zeros

The implementation expands the input, checks that it is polynomial in the supplied variable, obtains its coefficient array, and reduces each coefficient. It accepts a reduced coefficient only when `Element[c,Algebraics]` evaluates to `True` [10]. This means "recognized exact algebraic value," not "a symbolic formula that might become algebraic after assumptions are added."

Approximate real *and complex* values are rejected using `InexactNumberQ`; checking only for subexpressions with head `Real` is an unnecessarily fragile way to handle atomic complex values [13]. Ordinary evaluation can already eliminate an inexact term before the function is called, so the guarantee applies to the evaluated expression actually received.

After reduction, exact zeros are identified by `SameQ` with the integer 0. This is deliberately stronger than a numerical tolerance or an unresolved symbolic equality. Every arithmetic helper returns reduced coefficients. Leading zeros are trimmed only after reduction, so input degree and divisors reflect the algebraic polynomial rather than its unreduced surface syntax. Integers and rationals take a cheap path through the reducer. Algebraic arithmetic that fails to reduce is reported as a failure, not converted into a numerical calculation.

11.4 Implementation architecture

The kernel file has a small set of private layers. Coefficient conversion establishes the input invariant. Dense-array addition and multiplication preserve it. Horner composition supplies independent recomposition. Candidate recovery computes the formal-root recurrence; an attempt then builds powers and performs descending subtraction. Degree enumeration and recursion wrap that core. Finally the public interface converts arrays back to polynomials and constructs diagnostics.

All arrays are in ascending coefficient order. The zero polynomial is the one-element array `{0}`. A normalized degree-d right component is therefore an array of length $d + 1$, whose first entry is zero and whose last entry is one. During descending recovery, the residual array keeps length $n + 1$, preventing an out-of-range pivot after its leading coefficients have been canceled. Trimming occurs only at the end of that routine.

Failures use a package-private catch tag, so an unrelated user `Throw` is not intentionally swallowed. Enumeration memoization is localized to one call rather than retained indefinitely in global caches. No call is made to `Factor`, `FactorList`, `Solve`, numerical root finding, or the built-in `Decompose`. Exact algebraic arithmetic can, of course, use substantial internal kernel algorithms of its own; "factorization-free" refers to the polynomial-decomposition logic, not to every hidden operation inside an algebraic-number implementation.

11.5 Failure contracts

An invalid input is never represented by `{p}`, since that would be indistinguishable from a proved indecomposable polynomial. Instead the package returns `Failure[tag,association]`. Input validation uses the tags `InexactInput`, `NotPolynomial`, and `NonAlgebraicCoefficient`. Failed exact reduction is tagged `AlgebraicArithmetic`. Invalid degrees and output limits use `InvalidRightDegree` and `InvalidLimit`, respectively. An empty factor list uses `EmptyChain`,

and an unsupported calling form uses `Arguments`. The separate tag `EnumerationLimit` means partial success with an incomplete list.

An unsuccessful valid degree attempt is *not* a `Failure`: it is a normal association with `Decomposable -> False` and an exact residual. This distinction is useful in automated applications. External time limits, aborts, resource exhaustion, and user-interrupted computations are not mathematical rejection certificates. The package does not promise to turn every possible kernel interruption into a structured failure.

12 Validation, certificates, and reproducibility

12.1 What has actually been executed

The Wolfram connector was invoked during preparation, but its evaluator endpoint returned HTTP 404. There was no local Wolfram executable available. Accordingly, this distribution contains no claimed Wolfram-kernel pass report and no claim of a verified kernel-version compatibility matrix. Its Wolfram source has been reviewed and lexically checked, but lexical checking is not Wolfram parsing or evaluation.

An independent implementation in Python 3.13.5 with SymPy 1.14.0 *was* executed successfully. It uses exact rational and algebraic-number fields, its own coefficient-array routines, and no approximate zero tests. The machine-readable report is `validation/validation_results.json`. The executed checks were:

Validation category	Count
Motivating sextic and its degree-24 composition	2
Complete enumerations for powers and Chebyshev polynomials	6
Quintic algebraic coefficient example	1
Quintic coefficient together with the independent radical $\sqrt{3}$	1
Generated exact composites over six coefficient fields	144
Comparisons with the independent triangular candidate recurrence	72
Residual identities and pivot-zero checks over all candidate degrees	348
Members of the absolutely indecomposable family $x^n + x$	27
Comparisons with SymPy's separate rational decomposition routine	28
Constant, linear, and structural regression cases	6

These are overlapping categories of checks, not a sum of independent random trials. The report records the random seed 206618, interpreter versions, and elapsed time. The elapsed time is an observation for that validation run, not a benchmark of the Wolfram package.

The six fields for the generated composites are $\mathbb{Q}$, $\mathbb{Q}(\sqrt{2})$, $\mathbb{Q}(i)$, $\mathbb{Q}(\sqrt[3]{2})$, $\mathbb{Q}(\sqrt{2}, \sqrt{3})$, and $\mathbb{Q}(\alpha)$ with $\alpha^5 - \alpha - 1 = 0$. Every generated input is constructed from explicit outer and inner polynomials, including nonmonic inner leading coefficients and nonzero inner constants. Its expected normalized factors are therefore known algebraically. The separate mixed-field fixture in $\mathbb{Q}(\alpha, \sqrt{3})$ matches the article's `Root` example.

For each generated input the tests verify the recovered designated-degree pair, a complete chain, and the residual identity for every proper candidate degree. A second recurrence constructs candidates by direct leading-coefficient comparison rather than by the differential recurrence. The rational oracle comparisons check agreement of complete degree multisets and independently check the returned identities. These experiments support the mathematics and the algorithm design. They do not establish that the Wolfram implementation has no language-specific or kernel-specific defect.

12.2 The supplied Wolfram tests

The file `Tests/AlgebraicDecomposition.wlt` contains 53 `VerificationTest` specifications, one of which covers 12 generated radical composites. The tests include both user examples, polynomial `Root` coefficients, `AlgebraicNumber` input, complex algebraic coefficients, real and complex approximate-input rejection, algebraically zero leading coefficients, single-distinct-derivative-factor examples, nonunique complete chains, output limits, negative certificates, and convention/verification distinctions.

Run the suite from the extracted root directory with:

```
wolframscript -file Tests/RunTests.wls
```

Or evaluate in a Wolfram session:

```
TestReport["/path/to/algebraic-polynomial-decomposition/Tests/AlgebraicDecomposition.
    wlt"]
```

The runner prints the kernel version and test counts, exports a local `LastTestReport.wxf`, and exits with a nonzero status unless the report succeeds and exactly 53 tests are observed. It detects report properties instead of assuming that every kernel uses the same test-count interface [14]. A production adoption should preserve the actual kernel version and resulting test report. Test assertions are specifications until they have run; merely shipping them is not a test pass.

12.3 Reproducing the independent checks and rebuilding the article

With Python and SymPy installed, run:

```
python validation/reference_validation.py
python validation/check_wl_structure.py
```

The second command checks balanced delimiters, nested comments, and quoted strings in the Wolfram source files. Its JSON report explicitly states that scope. Neither command needs network access once its dependencies are installed.

The article is built from the `article` directory:

```
latexmk -pdf -interaction=nonstopmode -halt-on-error algebraic-decomposition.tex
```

A normal TeX installation with the packages listed in the preamble is sufficient. The source appendix reads the actual kernel file from the adjacent `Kernel` directory. Keep that structure when rebuilding; the printed source and distributed source then cannot silently diverge. Two runs of `pdflatex` can be used instead of `latexmk` when references are already stable, with additional runs if the log requests them.

12.4 Independent certificate checking in practice

For a successful pair, an external checker can expand $G(h) - p$ and reduce every coefficient to zero. For a rejected degree, checking only $p = G(h) + R$ with $R \neq 0$ is insufficient: an arbitrary wrong h could produce such a remainder even when another h decomposes p. A valid rejection checker must also verify that h is normalized of degree d and obeys the top-coefficient condition (4). Uniqueness then proves that no alternative right component was overlooked.

A complete certificate may therefore be organized as follows: for each accepted factor, retain its composition identity; for each claimed indecomposable nonlinear factor, retain the rejected

attempts at every proper degree; for each rejection, check normalization, top matching, residual recovery, and one nonzero coefficient. Prime degrees need only their degree argument. The package exposes the data needed to build such a certificate collection, but it does not serialize a standalone proof-assistant object or claim a formally verified arithmetic kernel.

13 Boundaries and directions for further development

13.1 Characteristic zero is essential to this implementation

The direct triangular recursion divides by the outer degree m. It extends to some positive-characteristic situations where m is invertible. The differential recurrence used by the package also divides by j, and therefore must not be transplanted unchanged into a finite field merely because m is invertible. This implementation exposes no `Modulus` option.

Wild characteristic can invalidate uniqueness itself. Over an algebraic closure of $\mathbb{F}_p$, let $b^{p+1} = 1$. Then

$$(t^p + b^p t) \circ (x^p - bx) = x^{p^2} - b^{p+1}x = x^{p^2} - x.$$

There are multiple distinct normalized degree-p right components $x^p - bx$ of the same polynomial. This does not contradict theorem 2.3: the outer degree p is zero in the field. It demonstrates why a characteristic-zero theorem cannot simply be relabeled as a theorem over arbitrary fields.

13.2 Symbolic parameters, approximations, and other objects

The mathematical algorithm makes sense over a symbolic rational-function field of characteristic zero, but a dependable implementation must track denominator nonvanishing, specializations where the degree drops, and exact zero identities in that field. This package deliberately rejects unassigned symbolic coefficients rather than treating generic answers as universally valid. Extending to symbolic parameters is a separate backend and specialization problem.

Approximate decomposition is also different. It requires a norm on coefficient space, a tolerance or statistical model, and a method for finding a nearby point on a decomposition locus. Exact arithmetic should not be weakened into such a method by changing `SameQ[coefficient,0]` into a tolerance comparison. The family $x^n + 10^{-100}x$ makes that distinction concrete.

Rational-function composition, multivariate functional decomposition, decomposition over specified integral subrings, and representation-minimization of radicals are outside the public API. The absence of these features is not an obstacle to the exact univariate algebraic-coefficient problem posed here.

13.3 Extensions supported by the theory

A fixed primitive-element arithmetic backend would address repeated conversion cost. Faster truncated power-series arithmetic and compositional right division would address the degree dependence. The right-component poset from section 7 suggests an alternative enumerator that computes all global right components once, builds divisibility cover edges, and enumerates maximal chains with reusable quotient certificates. None of these optimizations changes the finite candidate theorem or the acceptance criterion.

Formal verification could separate the polynomial-array invariant, the formal-root recurrence, descending recovery, and exact number-field arithmetic into distinct modules. The mathematical proofs here identify the necessary lemmas, but the present software is a reference implementation accompanied by proofs and tests, not a proof-assistant-verified library.

14 Conclusion

Radicals and algebraic `Root` objects do not require a different decomposition theory. They require exact arithmetic in the coefficient field and reliable equality to zero. Once the affine ambiguity is removed, a proposed right degree determines a single candidate from the leading coefficients. Exact recovery of the outer polynomial either produces the desired identity or a definitive residual rejection.

This yields a finite, factorization-free procedure for one complete decomposition, every normalized right pair, and every normalized complete chain. The components stay in the input coefficient field, so extending that field cannot cure a genuine failure of decomposition in characteristic zero. The supplied package implements these statements directly, documents its scope and failure modes, and includes both executable Wolfram test specifications and independently executed exact-field validation.

A Complete package source

The following is included directly from the distributed kernel file. The loader, examples, Wolfram tests, and independent Python validation are separate files in the archive.

```
(* ::Package:: *)
(* AlgebraicDecomposition 1.0.0 -- exact characteristic-zero decomposition.
   Original implementation supplied with the accompanying article.
   All coefficient arrays are in ASCENDING order. No Factor, Solve,
   numerical zero tests, or calls to the built-in Decompose are used. *)

BeginPackage["AlgebraicDecomposition`"];

AlgebraicDecompose::usage =
 "AlgebraicDecompose[p,x] returns one complete decomposition {f1,...,fr}, outermost first, with exact
     algebraic coefficients. All factors except the first are monic with zero constant term. Constants
     and linear inputs return {p}.";
AlgebraicDecompositions::usage =
 "AlgebraicDecompositions[p,x] returns all complete decompositions modulo insertion of affine maps, in
     normalized form. Option \"MaxDecompositions\" (default 1000) is a positive integer or Infinity. A
     limit failure contains partial output, never an assertion of completeness.";
AlgebraicRightDecompositions::usage =
 "AlgebraicRightDecompositions[p,x] returns every proper pair {g,h} with p=g(h), h monic and h[0]=0,
     sorted by degree of h. An empty list proves indecomposability when Degree[p]>=2.";
AlgebraicDecompositionAttempt::usage =
 "AlgebraicDecompositionAttempt[p,x,d] returns the unique normalized candidate of proper right degree d,
      its recovered outer polynomial, and an exact residual certificate. Invalid degrees return Failure.
     ";
ComposeAlgebraicPolynomials::usage =
 "ComposeAlgebraicPolynomials[{f1,...,fr},x] computes f1[f2[...fr[x]...]] by exact coefficient
     arithmetic. The input list must be nonempty.";
VerifyAlgebraicDecomposition::usage =
 "VerifyAlgebraicDecomposition[p,{f1,...,fr},x] independently recomposes the factors and returns exact
     identity, normalization, and indecomposability checks. Constants and linear polynomials are
     handled as singleton conventions, not as indecomposable nonlinear factors.";

Options[AlgebraicDecompositions] = {"MaxDecompositions" → 1000};

Begin["`Private`"];

(* A package-private catch tag does not intercept unrelated user Throws. *)
fail[tag_String, text_String, extra_: <||>] :=
 Throw[Failure[tag, Join[<|"MessageTemplate" → text|>, extra]], adFailureTag];

(* Cheap rational cases avoid algebraic-number conversion overhead. *)
rr[z_Integer] := z;
rr[z_Rational] := z;
rr[z_] := Module[{r},
 r = Quiet[Check[RootReduce[z], $Failed]];
 If[r === $Failed || !FreeQ[r, _RootReduce],
```

```
  fail["AlgebraicArithmetic", "Exact coefficient reduction did not complete.",
   <|"Expression" → z|>]];
 r
];

trim[v_List] := Module[{w = v},
 If[w === {}, Return[{0}]];
 While[Length[w] > 1 && Last[w] === 0, w = Most[w]];
 w
];

deg[v_List] := If[v === {0}, -Infinity, Length[v] - 1];

(* Validation takes place AFTER coefficient reduction and BEFORE degree tests.
   In particular, a high-degree coefficient may be algebraically zero. *)
coefficients[p_, x_Symbol] := Module[{q, c, r},
 If[!FreeQ[p, _?InexactNumberQ],
  fail["InexactInput", "Approximate numbers are not accepted; provide exact algebraic coefficients."]];
 q = Quiet[Check[Expand[p], $Failed]];
 If[q === $Failed || !TrueQ[PolynomialQ[q, x]],
  fail["NotPolynomial", "The input must be a univariate polynomial in the specified symbol."]];
 c = CoefficientList[q, x];
 If[c === {}, c = {0}];
 r = rr /@ c;
 If[!And @@ (TrueQ[Element[#, Algebraics]] & /@ r),
  fail["NonAlgebraicCoefficient", "Every coefficient must be a recognized exact algebraic number; free
     parameters are not supported.",
   <|"Coefficients" → r|>]];
 trim[r]
];

poly[v_List, x_Symbol] := Total[MapIndexed[#1 x^(First[#2] - 1) &, v]];

add[a_List, b_List] := trim[MapThread[rr[#1 + #2] &,
 {PadRight[a, Max[Length[a], Length[b]]],
  PadRight[b, Max[Length[a], Length[b]]]}]];

mul[a_List, b_List] := Module[{na, nb},
 If[a === {0} || b === {0}, Return[{0}]];
 na = Length[a] - 1; nb = Length[b] - 1;
 trim[Table[rr[Total[Table[a[[i + 1]] b[[k - i + 1]],
   {i, Max[0, k - nb], Min[k, na]}]]], {k, 0, na + nb}]]
];

(* Horner composition is separate from the candidate/recovery code. *)
compose[a_List, b_List] := Module[{r = {Last[a]}},
 Do[r = add[mul[r, b], {a[[j]]}], {j, Length[a] - 1, 1, -1}];
 r
];

composeChain[cs_List] := Fold[compose, First[cs], Rest[cs]];
properDegrees[n_] := If[!IntegerQ[n] || n < 4, {},
 Select[Divisors[n], 1 < # < n &]];

(* If n=m d and B(t)=p(1/t)t^n/lc(p), compute U=B^(1/m) mod t^d.
   From B U'=(1/m) U B':
   u_j = (1/j) Sum[((1+1/m)i-j) b_i u_(j-i), {i,1,j}].
   The constant coefficient of h is FIXED at zero, not taken from U. *)
candidate[c_List, d_Integer] := Module[{n, m, b, u},
 n = Length[c] - 1; m = Quotient[n, d];
 b = Table[rr[c[[n - i + 1]]/Last[c]], {i, 1, d - 1}];
 u = ConstantArray[0, d]; u[[1]] = 1;
 Do[u[[j + 1]] = rr[Total[Table[
    ((1 + 1/m) i - j) b[[i]] u[[j - i + 1]],
    {i, 1, j}]]/j], {j, 1, d - 1}];
 Prepend[Reverse[u], 0]
];

(* Build h^0,...,h^m once. Descending subtraction extracts the only
   possible g. A nonzero residual is a proof of nonexistence for this d. *)
attempt[c_List, d_Integer] := Module[{n, m, h, powers, r, g, a},
 n = Length[c] - 1; m = Quotient[n, d];
 h = candidate[c, d];
```

```
 powers = NestList[mul[#, h] &, {1}, m];
 r = c; g = ConstantArray[0, m + 1];
 Do[
  a = r[[k d + 1]]; g[[k + 1]] = a;
  If[a =!= 0,
   r = MapThread[rr[#1 - a #2] &,
     {r, PadRight[powers[[k + 1]], n + 1]}]],
  {k, m, 0, -1}];
 {trim[g], h, trim[r]}
];

(* Minimal successful right degree implies an indecomposable right factor. *)
firstSplit[c_List] := Module[{r},
 Do[r = attempt[c, d]; If[r[[3]] === {0}, Return[Take[r, 2]]],
  {d, properDegrees[deg[c]]}];
 None
];

oneChain[c_List] := Module[{s = firstSplit[c]},
 If[s === None, {c}, Append[oneChain[s[[1]]], s[[2]]]]
];

rightPairs[c_List] := Module[{ans = {}, r},
 Do[r = attempt[c, d];
  If[r[[3]] === {0}, AppendTo[ans, Take[r, 2]]],
  {d, properDegrees[deg[c]]}];
 ans
];

attemptRecord[c_List, d_Integer, x_Symbol] := Module[{a, w, r},
 a = attempt[c, d]; r = a[[3]];
 w = If[r === {0}, Missing["NotApplicable"],
  <|"Exponent" → deg[r], "Coefficient" → Last[r]|>];
 <|"InputDegree" → deg[c], "RightDegree" → d,
   "OuterDegree" → Quotient[deg[c], d],
   "Decomposable" → (r === {0}),
   "Outer" → poly[a[[1]], x], "Inner" → poly[a[[2]], x],
   "Residual" → poly[r, x], "NonzeroWitness" → w|>
];

AlgebraicDecompose[p_, x_Symbol] := Catch[Module[{c},
 c = coefficients[p, x]; poly[#, x] & /@ oneChain[c]
], adFailureTag];

AlgebraicRightDecompositions[p_, x_Symbol] := Catch[Module[{c},
 c = coefficients[p, x]; Map[poly[#, x] &, rightPairs[c], {2}]
], adFailureTag];

AlgebraicDecompositionAttempt[p_, x_Symbol, d_Integer] := Catch[Module[{c},
 c = coefficients[p, x];
 If[!MemberQ[properDegrees[deg[c]], d],
  fail["InvalidRightDegree", "The right degree must be a proper divisor of the reduced input degree,
     strictly between 1 and that degree.",
   <|"InputDegree" → deg[c], "RightDegree" → d|>]];
 attemptRecord[c, d, x]
], adFailureTag];

AlgebraicDecompositions[p_, x_Symbol, OptionsPattern[]] := Catch[
 Module[{c, limit = OptionValue["MaxDecompositions"], all, indec, result},
  If[!(limit === Infinity || (IntegerQ[limit] && limit > 0)),
   fail["InvalidLimit", "MaxDecompositions must be a positive integer or Infinity."]];
  c = coefficients[p, x];
  (* Memoization is local to this call. Returning at most limit+1 items
     suffices to detect truncation, including in recursive subproblems. *)
  indec[v_List] := indec[v] = (firstSplit[v] === None);
  all[v_List] := all[v] = Module[{pairs, resultHere = {}, tailChains},
   pairs = rightPairs[v];
   If[pairs === {}, Return[{{v}}]];
   Do[If[indec[pair[[2]]],
     tailChains = all[pair[[1]]];
     Do[
      AppendTo[resultHere, Append[ch, pair[[2]]]];
      If[Length[resultHere] > limit, Return[resultHere]],
```

```
      {ch, tailChains}]],
    {pair, pairs}];
   resultHere
  ];
  result = all[c];
  If[Length[result] > limit,
   fail["EnumerationLimit", "The requested enumeration limit was exceeded. PartialDecompositions is not
      an exhaustive list.",
    <|"Limit" → limit, "Complete" → False,
      "PartialDecompositions" → Map[poly[#, x] &, Take[result, limit], {2}]|>]];
  Map[poly[#, x] &, result, {2}]
 ], adFailureTag];

ComposeAlgebraicPolynomials[ps_List, x_Symbol] := Catch[Module[{cs},
 If[ps === {}, fail["EmptyChain", "The composition list must be nonempty."]];
 cs = coefficients[#, x] & /@ ps;
 poly[composeChain[cs], x]
], adFailureTag];

VerifyAlgebraicDecomposition[p_, ps_List, x_Symbol] := Catch[
 Module[{c, cs, residual, identity, normalized, nonlinear, primeFactors,
   convention},
  If[ps === {}, fail["EmptyChain", "The composition list must be nonempty."]];
  c = coefficients[p, x]; cs = coefficients[#, x] & /@ ps;
  residual = add[c, -composeChain[cs]];
  identity = (residual === {0});
  normalized = And @@ ((Last[#] === 1 && First[#] === 0) & /@ Rest[cs]);
  nonlinear = And @@ (deg[#] >= 2 & /@ cs);
  primeFactors = nonlinear && And @@ (firstSplit[#] === None & /@ cs);
  convention = deg[c] < 2 && Length[cs] == 1 && identity;
  <|"IdentityVerified" → identity, "Residual" → poly[residual, x],
    "Normalized" → normalized,
    "AllFactorsIndecomposable" → primeFactors,
    "ValidCompleteDecomposition" → (identity && primeFactors),
    "ValidConstantOrLinearConvention" → convention,
    "Degrees" → (deg /@ cs)|>
 ], adFailureTag];

(* Fallbacks make invalid calling forms explicit rather than unevaluated. *)
AlgebraicDecompose[___] := Failure["Arguments", <|"MessageTemplate" → "Use AlgebraicDecompose[p,x]
    with an unassigned symbol x."|>];
AlgebraicRightDecompositions[___] := Failure["Arguments", <|"MessageTemplate" → "Use
    AlgebraicRightDecompositions[p,x]."|>];
AlgebraicDecompositionAttempt[___] := Failure["Arguments", <|"MessageTemplate" → "Use
    AlgebraicDecompositionAttempt[p,x,d] with an integer d."|>];
AlgebraicDecompositions[___] := Failure["Arguments", <|"MessageTemplate" → "Use
    AlgebraicDecompositions[p,x,options]."|>];
ComposeAlgebraicPolynomials[___] := Failure["Arguments", <|"MessageTemplate" → "Use
    ComposeAlgebraicPolynomials[nonemptyList,x]."|>];
VerifyAlgebraicDecomposition[___] := Failure["Arguments", <|"MessageTemplate" → "Use
    VerifyAlgebraicDecomposition[p,nonemptyList,x]."|>];

End[];
EndPackage[];
```

References

[1] Vladimir Reshetnikov, "Is it possible to make Decompose work with coefficients containing radicals?" Mathematica Stack Exchange, 2019; answer by Greg Hurst and historical comment by Daniel Lichtblau. https://mathematica.stackexchange.com/q/206618. The user supplied the question, answer code, and comment that motivated this article.

[2] Dexter Kozen and Susan Landau, "Polynomial decomposition algorithms," *Journal of Symbolic Computation* **7** (1989), 445–456. doi:10.1016/S0747-7171(89)80027-6. Author publication summary: https://www.cs.cornell.edu/~kozen/Papers/papers_by_year.htm.

[3] Joachim von zur Gathen, "Functional decomposition of polynomials: the tame case," *Journal of Symbolic Computation* **9** (1990), 281–299. doi:10.1016/S0747-7171(08)80014-4.

[4] Mark Giesbrecht, *Some Results on the Functional Decomposition of Polynomials*, Master's thesis, University of Toronto, 1988. Electronic version: arXiv:1004.5433 (2010), especially Chapter 2. https://arxiv.org/abs/1004.5433.

[5] Michael E. Zieve and Peter Müller, "On Ritt's polynomial decomposition theorems," arXiv:0807.3578 (2008). https://arxiv.org/abs/0807.3578.

[6] Brian K. Wyman and Michael E. Zieve, "Two questions on polynomial decomposition," arXiv:1301.5211 (2013). https://arxiv.org/abs/1301.5211.

[7] Wolfram Research, *Decompose*, Wolfram Language documentation, accessed 7 September 2026. https://reference.wolfram.com/language/ref/Decompose.html.

[8] Wolfram Research, *Root*, Wolfram Language documentation, accessed 7 September 2026. https://reference.wolfram.com/language/ref/Root.html.

[9] Wolfram Research, *RootReduce*, Wolfram Language documentation, accessed 7 September 2026. https://reference.wolfram.com/language/ref/RootReduce.html.

[10] Wolfram Research, *Algebraics*, Wolfram Language documentation, accessed 7 September 2026. https://reference.wolfram.com/language/ref/Algebraics.html.

[11] Wolfram Research, *AlgebraicNumber*, Wolfram Language documentation, accessed 7 September 2026. https://reference.wolfram.com/language/ref/AlgebraicNumber.html.

[12] Wolfram Research, *ToNumberField*, Wolfram Language documentation, accessed 7 September 2026. https://reference.wolfram.com/language/ref/ToNumberField.html.

[13] Wolfram Research, *InexactNumberQ*, Wolfram Language documentation, accessed 7 September 2026. https://reference.wolfram.com/language/ref/InexactNumberQ.html.

[14] Wolfram Research, *TestReportObject*, Wolfram Language documentation, accessed 7 September 2026. https://reference.wolfram.com/language/ref/TestReportObject.html.